

Aide-mémoire M1 — Entraîner & évaluer un modèle *proprement*

Le fil rouge du module, en une page · Marianne A. · classification supervisée sur données tabulaires

L'objectif du M1. Partir d'un CSV brut et produire un modèle dont la performance est *honnête et reproductible* — pas un chiffre gonflé. Tout le module tient dans une règle : **on n'apprend que sur le train, on évalue une seule fois sur un test scellé, et on juge sur la bonne métrique**. Le livrable final : `experiments.md` (ce qu'on a essayé), `verdict.md` (bat-on la baseline ?) et `joblib.dump` (le pipeline versionné).

Repère mental Un bon score sur le train ne veut rien dire. La seule question qui compte : « *comment le modèle se comporte-t-il sur des données qu'il n'a jamais vues ?* »

1 Le workflow, étape par étape

1 EDA — comprendre les données

Quoi : `df.info()`, `df.describe()`, valeurs manquantes, types. **Pourquoi :** identifier la **cible** et repérer le **déséquilibre des classes** — ça décide de la métrique.

```
df.info(); df.describe()
df["cible"].value_counts(normalize=True) # déséquilibre ?
```

2 Séparer train / test — *tout de suite*

Quoi : `train_test_split` avec `stratify=y` et `random_state=42`. **Pourquoi :** le test (~20 %) est **scellé** — on n'y touche qu'à la toute fin, une seule fois.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)
```

3 Construire le pipeline

Quoi : un `ColumnTransformer` (Scaler pour le numérique, `OneHotEncoder` pour le catégoriel) + le modèle, le tout dans un `Pipeline`. **Pourquoi :** le préprocessing voyage **avec** le modèle → pas de fuite.

```
pre = ColumnTransformer([
    ("num", StandardScaler(), num_cols),
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols)])
pipe = Pipeline([("pre", pre), ("clf",
    RandomForestClassifier(random_state=42))])
```

4 Entraîner — `.fit()` sur le train seul

Quoi : `pipe.fit(X_train, y_train)`. **Pourquoi :** le scaler et l'OHE apprennent leurs paramètres **uniquement** sur le train. Le test reste inconnu.

```
pipe.fit(X_train, y_train) # le test n'est PAS vu ici
```

5 Prédire sur le test scellé

Quoi : `pipe.predict(X_test)`. **Pourquoi :** c'est le **seul et unique** contact avec le test — il simule la vraie vie.

```
y_pred = pipe.predict(X_test) # test touché 1 SEULE fois
```

7 Verdict & versionner

Quoi : bat-on la **baseline** ? On écrit `experiments.md` + `verdict.md` et on sauvegarde le pipeline. **Pourquoi :** résultat argumenté et **reproductible**.

```
joblib.dump(pipe, "model.joblib") # le pipeline entier
```

6 Évaluer avec la bonne métrique

Quoi : `classification_report`, **F1 macro**, matrice de confusion — **PAS** l'accuracy seule. **Pourquoi :** en classes déséquilibrées, l'accuracy ment (voir bloc 2).

```
print(classification_report(y_test, y_pred))
f1_score(y_test, y_pred, average="macro")
```

★ La règle d'or

Le test set est **sacré**. On le sépare en premier, on ne le regarde qu'à la fin, une seule fois. Tout réglage (hyperparamètres, choix de modèle) se fait par **validation croisée sur le train** — jamais en regardant le test.

2 Métriques & évaluation — laquelle, quand ?

MÉTRIQUE	CE QUE ÇA MESURE	QUAND L'UTILISER	PIÈGE / À RETENIR
Accuracy <code>accuracy_score</code>	Proportion de prédictions correctes, toutes classes confondues.	Seulement si les classes sont équilibrées et ont la même importance.	Trompeuse en déséquilibre : 95 % de « non-défaut » → 95 % d'accuracy en prédisant toujours « non ». Inutile.
Précision <code>precision (classe)</code>	Parmi les cas prédits positifs , combien le sont vraiment.	Quand un faux positif coûte cher (ex. bloquer un bon client).	Monte quand on prédit « positif » avec prudence. À lire par classe.
Rappel (recall) <code>recall (classe)</code>	Parmi les vrais positifs , combien on a réussi à retrouver.	Quand rater un positif coûte cher (défaut de crédit, maladie, fraude).	Souvent la métrique clé du M1 sur la classe minoritaire (ex. « défaut »).
F1-score <code>f1 (classe)</code>	Moyenne harmonique précision × rappel : un seul chiffre d'équilibre.	Quand on veut un compromis précision/rappel sur une classe.	Pénalise fort si l'une des deux est basse (contrairement à une moyenne simple).
F1 macro <code>average="macro"</code>	Moyenne des F1 de chaque classe , à poids égal.	Le réflexe en classes déséquilibrées : la classe rare pèse autant que la fréquente.	C'est la métrique de verdict recommandée ici. À ne pas confondre avec F1 <i>weighted</i> (repondéré par effectif).

MÉTRIQUE	CE QUE ÇA MESURE	QUAND L'UTILISER	PIÈGE / À RETENIR
Matrice de confusion <code>confusion_matrix</code>	Le tableau VP / FP / FN / VN : d'où viennent les erreurs.	Toujours — c'est la source de toutes les autres métriques.	Regarder où le modèle se trompe (fausses alertes vs cas ratés), pas juste le score global.
classification_report <code>classification_report</code>	précision / rappel / F1 par classe + moyennes macro et weighted.	La vue d'ensemble à sortir systématiquement après <code>predict</code> .	Lire ligne par ligne : la moyenne globale peut cacher une classe catastrophique.

3 Pipeline sklearn & fuite de données

Pourquoi un Pipeline et pas des étapes séparées ? Parce qu'il garantit que le préprocessing (scaling, encodage) apprend ses paramètres **uniquement sur le train**, puis les *applique* au test — sans jamais l'utiliser pour apprendre. C'est exactement ce qui empêche la **fuite de données** (data leakage).

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.pipeline import Pipeline

pre = ColumnTransformer([
    ("num", StandardScaler(), num_cols), # centre-réduit le numérique
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols), # encode le catégoriel
])
pipe = Pipeline([("pre", pre), ("clf", RandomForestClassifier(random_state=42))])

pipe.fit(X_train, y_train) # apprend TOUT (scaler+OHE+modèle) sur le train seul
y_pred = pipe.predict(X_test) # applique au test, sans rien réapprendre
joblib.dump(pipe, "model.joblib") # on versionne le pipeline ENTIER, pas juste le modèle
```

La fuite de données en une phrase. Si tu fais `scaler.fit(X)` ou `OneHotEncoder().fit(X)` sur *tout* le jeu **avant** le split (ou si tu réutilises des infos du test pour préparer les données), ton score sera **faussement excellent** et s'effondrera en production. Le pipeline + « fit sur le train seul » est le garde-fou.

4 Pièges & erreurs fréquentes

LE PIÈGE	SYMPTÔME	LE CORRECTIF
Préprocessing avant le split	Score de test excellent mais irréaliste ; chute en prod.	Tout mettre dans un Pipeline et ne <code>fit()</code> que sur le train (fuite de données).
Évaluer sur le train	Métriques quasi parfaites, trop belles pour être vraies.	Toujours évaluer sur le test scellé (ou en validation croisée), jamais sur les données d'apprentissage.
Toucher / régler sur le test	Le modèle « marche » en TP mais généralise mal.	Tuning et choix de modèle par validation croisée sur le train . Le test ne sert qu'au verdict final, une fois.
Se fier à l'accuracy	95 % d'accuracy... en ne détectant aucun « défaut ».	Regarder F1 macro et le rappel de la classe minoritaire , pas le score global.
Oublier <code>stratify=y</code>	Proportions de classes différentes entre train et test ; scores instables.	Ajouter <code>stratify=y</code> au <code>train_test_split</code> (surtout en déséquilibre).
Catégorie inconnue en test	<code>ValueError</code> au moment du <code>predict</code> .	<code>OneHotEncoder(handle_unknown="ignore")</code> .
Résultat non reproductible	Un score différent à chaque exécution ; comparaisons impossibles.	Fixer <code>random_state=42</code> partout (split + modèle) et tracer dans <code>experiments.md</code> .
Comparer sur des splits différents	« Mon modèle bat le tien » alors que les jeux ne sont pas les mêmes.	Même split / même seed pour tout le monde, ou validation croisée identique. Baseline commune à battre.

À retenir. Sépare le test en premier et scelle-le · préprocessing dans un Pipeline, `fit` sur le train seul · juge sur F1 macro + rappel de la classe rare, jamais l'accuracy seule · fixe les seeds et **Fiche 5/12 · Série ML** documente (`experiments.md`, `verdict.md`, `joblib.dump`). Un résultat honnête et reproductible vaut mieux qu'un chiffre flatteur.